

Borrow a network that can see, then change what it looks at

The notebook is written and works. A pretrained image network has its last layer replaced and retrained on two classes. Your job is the data: run it, then point it at a different pair of classes or at your own photographs.

Guided lab

25 minutes

PyTorch · torchvision

GPU helps · Colab is fine

Before you start 1 · the pretrained backbone 2 · choose the data 3 · fine-tune the new layer
4 · your runs Reflection If you finish early

Before you start

Open the provided `lab16.ipynb`, ideally on Google Colab with a GPU runtime. The first run downloads CIFAR-10 and MobileNetV3-Small weights, so start it before the lecture ends.



By the end of this lab you can

- Explain what is frozen and what is trained when a pretrained backbone is adapted.
- Run a fine-tuning loop and read its validation accuracy.
- Change the dataset — two new classes, or a local folder of images — and retrain.

STEP 1 · GIVEN

the pretrained backbone

1

MobileNetV3-Small was trained on 1.2 million ImageNet photographs. We keep everything it learned about edges, textures and shapes, and replace only its final layer with a new one that has two outputs.

given

PYTHON

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
weights = models.MobileNet_V3_Small_Weights.DEFAULT
transform = weights.transforms()

model = models.mobilenet_v3_small(weights=weights)
for p in model.features.parameters():
    p.requires_grad = False # frozen: the borrowed knowledge
model.classifier[-1] = nn.Linear(model.classifier[-1].in_features, 2) # new: our two
classes
model = model.to(device)

trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print("device:", device, f" trainable parameters: {trainable:,} of {total:,}")
```

STEP 2 · 3 MINUTES

choose the data

2

This is the cell you change. CIFAR-10 class numbers: 0 airplane, 1 automobile, 2 bird, 3 cat, 4 deer, 5 dog, 6 frog, 7 horse, 8 ship, 9 truck.

To use **your own photographs** instead, put them in `data/local_images/<class_a>/` and `data/local_images/<class_b>/` — about 40 images each is enough — and set `USE_LOCAL_FOLDER = True`.

```

USE_LOCAL_FOLDER = False
CLASSES = (3, 5)          # TODO: after the first run, change to a different pair
PER_CLASS = 200

if USE_LOCAL_FOLDER:
    full = datasets.ImageFolder("data/local_images", transform=transform)
    n_val = len(full) // 4
    train_ds, val_ds = torch.utils.data.random_split(full, [len(full) - n_val, n_val],
generator=torch.Generator().manual_seed(7))
    remap = lambda labels: labels
else:
    train_full = datasets.CIFAR10("./data", train=True, download=True, transform=transform)
    val_full = datasets.CIFAR10("./data", train=False, download=True, transform=transform)
    def take(ds, n):
        counts, idx = {c: 0 for c in CLASSES}, []
        for i, t in enumerate(ds.targets):
            if t in counts and counts[t] < n:
                idx.append(i); counts[t] += 1
        return Subset(ds, idx)
    train_ds, val_ds = take(train_full, PER_CLASS), take(val_full, PER_CLASS // 2)
    lookup = {c: k for k, c in enumerate(CLASSES)}
    remap = lambda labels: torch.tensor([lookup[int(t)] for t in labels])

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=64)
print("training images:", len(train_ds), " validation images:", len(val_ds))

```

STEP 3 · GIVEN

fine-tune the new layer

3

Three epochs, cross-entropy, Adam, and only the parameters with `requires_grad=True`. The loop has the same four lines at its heart as Lab 12's PyTorch cell.

```
def evaluate():
    model.eval(); correct = n = 0
    with torch.no_grad():
        for xb, yb in val_loader:
            yb = remap(yb).to(device)
            correct += (model(xb.to(device)).argmax(1) == yb).sum().item(); n += len(yb)
    return correct / n

loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam([p for p in model.parameters() if p.requires_grad], lr=1e-3)
print("before training, validation accuracy:", round(evaluate(), 3))
for epoch in range(3):
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device), remap(yb).to(device)
        opt.zero_grad(); loss = loss_fn(model(xb), yb); loss.backward(); opt.step()
    print(f"epoch {epoch + 1}: last batch loss {loss.item():.3f}    validation accuracy {evaluate():.3f}")
```

STEP 4 • 10 MINUTES

your runs

4

Run Steps 2–3 once as provided, then **at least once with different data**. Record:

Data	Training images	Accuracy before	Accuracy after 3 epochs
CIFAR-10 cat vs dog (provided)	400		
Your choice:			

Which pair was easier, and is that what you would have guessed from how the classes look?

Reflection



Write it in the notebook, in your own words

Answer in two or three sentences each:

1. What knowledge is being transferred, and where in `model` does it live?
2. Why can 400 images be enough here, when session 16 needed 1,347 just for even-versus-odd digits?
3. Why might unfreezing every layer on day one be risky with so little data?

Your answer:

If you finish early

A. Unfreeze only the last block — `for p in model.features[-1].parameters(): p.requires_grad = True`
— rebuild the optimiser with `lr=1e-4`, and train one more epoch. Did the extra freedom help or hurt?

your turn

PYTHON

TODO (optional extension A)